Mini project

SC42155: Modelling of Dynamical systems

by

Kira Benton & Per van der Klauw

Student numbers: 5261082 & 6378080

TU Delft

1

System Dynamics

1.1. Position and Velocity of Head (Q1.1)

To begin, the position of the head (mass) is found by assuming that the change in x and y is related to the change in cable length released by the left and right motors. The positive (upwards) change in cable length by motor l is defined as ΔA and the positive (downwards) change in cable length by motor r is defined as ΔB. It is assumed that the cables are not stretchable and do not slip. Next, a derivation for the system kinematics is as follows:

$$\Delta A = \Delta x + \Delta y \tag{1.1}$$

$$\Delta B = \Delta x - \Delta y \tag{1.2}$$

Adding and subtracting these equations from one another reveals the horizontal and vertical displacements.

$$\Delta y = (\Delta A - \Delta B)/2 \tag{1.3}$$

$$\Delta x = (\Delta A + \Delta B)/2 \tag{1.4}$$

Assuming a constant motor pulley radius r_m, the belt displacements relate to the motor angles by: $\Delta A = r_m\theta_l$ and $\Delta B = r_m\theta_r$. Assuming the print-head position and motor angles begin at zero, the position equations for the print head are:

$$x(\theta_l, \theta_r) = 1/2(r_m\theta_l + r_m\theta_r) \tag{1.5}$$

$$y(\theta_l, \theta_r) = 1/2(r_m\theta_l - r_m\theta_r) \tag{1.6}$$

To find the velocity of the head, the derivative of the position is taken as shown below:

$$\dot{x}(\dot{\theta_l}, \dot{\theta_r}) = 1/2(r_m\dot{\theta_l} + r_m\dot{\theta_r}) \tag{1.7}$$

$$\dot{y}(\dot{\theta_l}, \dot{\theta_r}) = 1/2(r_m\dot{\theta_l} - r_m\dot{\theta_r}) \tag{1.8}$$

These equations show that the horizontal movement of the print head is determined by summing the motor rotations, and the vertical movement is determined by taking their difference.

1.2. System State Space (Q1.2)

The system state space is derived from the system dynamics for each motor as shown below. The inductor voltage is defined as u_L and motor voltage is u_m.

1.2.1. Electrical Equations

The electrical dynamics are the same for both the left and right motors, thus the following equations are derived for a single motor using u_m, i_m, and θ and will later be applied to the individual l and r motors.
Resistor:

$$R_m i_m = u_R \tag{1.9}$$

Inductor:

$$u_L = L_m di_m/dt \tag{1.10}$$

Kirchoff's voltage equation:

$$u_m = L_m di_m/dt + R_m i_m + K_m\dot{\theta} \tag{1.11}$$

It is chosen to incorporate back-emf in order to be thorough and prior experience with motor equations always included it.

$$di_m/dt = u_m/L_m - K_m\dot{\theta}/L_m - R_m i_m/L_m \tag{1.12}$$

1.2.2. Motor equations

The torque generated by each motor is proportional to the motor current:

$$\tau_m = K_m i_m \tag{1.13}$$

Because the bearings contain rolling friction, this must be accounted for in the inertia equation.

$$J_m\ddot{\theta} + \tau_f + \tau_{belt} = \tau_m \tag{1.14}$$

The bearing friction is modelled as a drag force proportional to the tangential velocity of the motor pulley:

$$F_f = \mu_r v = \mu_r r_m \dot{\theta} \tag{1.15}$$

Torque is the product of force and pulley radius, so the friction torque becomes:

$$\tau_f = F_f r_m = \mu_r r_m^2 \dot{\theta} \tag{1.16}$$

Substituting in the friction torque and motor torque, the new inertia equation becomes:

$$J_m\ddot{\theta} + \mu_r r_m^2 \dot{\theta} + \tau_{belt} = K_m i_m \tag{1.17}$$

τ_{belt} is the torque the belt exerts on the motor shaft and is later broken up into τ_l & τ_r

1.2.3. Mechanical Equations

The forces and the x and y directions must incorporate the linear bearing friction.

$$F_x = m\ddot{x} + \mu_{l,x}\dot{x} \tag{1.18}$$

$$F_y = m\ddot{y} + \mu_{l,y}\dot{y} \tag{1.19}$$

Utilizing the power theory:

$$P_{in} = P_{out} \tag{1.20}$$

$$\tau_l * \dot{\theta_l} + \tau_r * \dot{\theta_r} = F_x\dot{x} + F_y\dot{y} \tag{1.21}$$

Substituting Eqs. (1.7) (1.8)

$$\tau_l * \dot{\theta_l} + \tau_r * \dot{\theta_r} = F_x(r_m/2(\dot{\theta_l} + \dot{\theta_r})) + F_y(r_m/2(\dot{\theta_l} - \dot{\theta_r})) \tag{1.22}$$

$$\tau_l * \dot{\theta_l} + \tau_r\dot{\theta_r} = (r_m/2(F_x + F_y)\dot{\theta_l}) + (r_m/2(F_x - F_y)\dot{\theta_r}) \tag{1.23}$$

The coefficients for $\dot{\theta_l}$ or $\dot{\theta_r}$ on either side of the equals sign must equate one another.

$$\tau_l = r_m/2(F_x + F_y) \tag{1.24}$$

$$\tau_r = r_m/2(F_x - F_y) \tag{1.25}$$

Then substituting values for F_x and F_y.

$$\tau_l = r_m/2(m\ddot{x} + \mu_{l,x}\dot{x} + m\ddot{y} + \mu_{l,y}\dot{y}) \tag{1.26}$$

$$\tau_r = r_m/2(m\ddot{x} + \mu_{l,x}\dot{x} - (m\ddot{y} + \mu_{l,y}\dot{y})) \tag{1.27}$$

Substituting in the derivatives of Eqs. (1.7) (1.8).

$$\tau_l = mr_m^2/4(\ddot{\theta_l} + \ddot{\theta_r} + \ddot{\theta_l} - \ddot{\theta_r}) + r_m^2/4\mu_{l,x}(\dot{\theta_l} + \dot{\theta_r}) + r_m^2/4\mu_{l,y}(\dot{\theta_l} - \dot{\theta_r}) \tag{1.28}$$

$$\tau_l = mr_m^2\ddot{\theta_l}/2 + r_m^2/4(\mu_{l,x} + \mu_{l,y})\dot{\theta_l} + r_m^2/4(\mu_{l,x} - \mu_{l,y})\dot{\theta_r} \tag{1.29}$$

$$\tau_r = mr_m^2/4(\ddot{\theta_l} + \ddot{\theta_r} - \ddot{\theta_l} + \ddot{\theta_r}) + r_m^2/4\mu_{l,x}(\dot{\theta_l} + \dot{\theta_r}) - r_m^2/4\mu_{l,y}(\dot{\theta_l} - \dot{\theta_r}) \tag{1.30}$$

$$\tau_r = mr_m^2\ddot{\theta_r}/2 + r_m^2/4(\mu_{l,x} - \mu_{l,y})\dot{\theta_l} + r_m^2/4(\mu_{l,x} + \mu_{l,y})\dot{\theta_r} \tag{1.31}$$

Substituting Eqs. (1.29) & (1.31) back into the motor equations.

$$K_m i_{m,l} = (J_m + mr_m^2/2)\ddot{\theta_l} + [\mu_r r_m^2 + r_m^2/4(\mu_{l,x} + \mu_{l,y})]\dot{\theta_l} + [r_m^2/4(\mu_{l,x} - \mu_{l,y}])\dot{\theta_r} \tag{1.32}$$

$$K_m i_{m,r} = (J_m + mr_m^2/2)\ddot{\theta_r} + [\mu_r r_m^2 + r_m^2/4(\mu_{l,x} + \mu_{l,y})]\dot{\theta_r} + [r_m^2/4(\mu_{l,x} - \mu_{l,y}])\dot{\theta_l} \tag{1.33}$$

1.2.4. State-Space

The equations will be simplified using the following substitutions before combining everything into the state-space:

$J_{eq} = J_m + mr_m^2/2,$
$b_1 = \mu_r r_m^2 + r_m^2/4(\mu_{l,x} + \mu_{l,y})$
$b_2 = r_m^2/4(\mu_{l,x} - \mu_{l,y})$

$$\begin{bmatrix} \dot{\theta}_l \\ \dot{\theta}_r \\ \ddot{\theta}_l \\ \ddot{\theta}_r \\ \frac{di_{m,l}}{dt} \\ \frac{di_{m,r}}{dt} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{-b_1}{J_{eq}} & \frac{-b2}{J_{eq}} & \frac{K_m}{J_{eq}} & 0 \\ 0 & 0 & \frac{-b_2}{J_{eq}} & \frac{-b_1}{J_{eq}} & 0 & \frac{K_m}{J_{eq}} \\ 0 & 0 & -\frac{K_m}{L_m} & 0 & -\frac{R_m}{L_m} & 0 \\ 0 & 0 & 0 & -\frac{K_m}{L_m} & 0 & -\frac{R_m}{L_m} \end{bmatrix} \begin{bmatrix} \theta_l \\ \theta_r \\ \dot{\theta}_l \\ \dot{\theta}_r \\ i_{m,l} \\ i_{m,r} \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ \frac{1}{L_m} & 0 \\ 0 & \frac{1}{L_m} \end{bmatrix} \begin{bmatrix} u_l \\ u_r \end{bmatrix}$$

1.3. Decoupling (Q1.3)

In order to convert a 6x6 matrix into a pair of 3x3 matrices, a Similarity Transform is conducted where $\bar{A} = P^{-1}AP$. In order to find P^{-1}, a z-matrix is created such that z = $[x, \dot{x}, i_x, y, \dot{y}, i_y]^T$ and z = $P^{-1}x_m$ where $x_m = \begin{bmatrix} \theta_l & \theta_r & \dot{\theta}_l & \dot{\theta}_r & i_{m,l} & i_{m,r} \end{bmatrix}^T$ i_x and i_y are found in a similar way to x and y in Q1.1 and are defined as:

$$i_x = 1/2(i_{m,l} + i_{m,r}) \tag{1.34}$$

$$i_y = 1/2(i_{m,l} - i_{m,r}) \tag{1.35}$$

$$P^{-1} = \begin{bmatrix} \frac{r_m}{2} & \frac{r_m}{2} & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{r_m}{2} & \frac{r_m}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{2} & \frac{1}{2} \\ \frac{r_m}{2} & -\frac{r_m}{2} & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{r_m}{2} & -\frac{r_m}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

By rearranging the z formula, x_m = Pz. Also, adding or subtracting the equations for x & y yields the following formulas for θ_l&θ_r.

$$\theta_l = x/r_m + y/r_m \tag{1.36}$$

$$\theta_r = x/r_m - y/r_m \tag{1.37}$$

$$P = \begin{bmatrix} \frac{1}{r_m} & 0 & 0 & \frac{1}{r_m} & 0 & 0 \\ \frac{1}{r_m} & 0 & 0 & -\frac{1}{r_m} & 0 & 0 \\ 0 & \frac{1}{r_m} & 0 & 0 & \frac{1}{r_m} & 0 \\ 0 & \frac{1}{r_m} & 0 & 0 & -\frac{1}{r_m} & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & -1 \end{bmatrix}$$

Next, matrix multiplication is completed and yields the following results.

$$P^{-1}A = \begin{bmatrix} 0 & 0 & \frac{r_m}{2} & \frac{r_m}{2} & 0 & 0 \\ 0 & 0 & \frac{-r_m(b_1+b_2)}{2J_{eq}} & \frac{-r_m(b_1+b_2)}{2J_{eq}} & \frac{r_m K_m}{2J_{eq}} & \frac{r_m K_m}{2J_{eq}} \\ 0 & 0 & -\frac{K_m}{2L_m} & -\frac{K_m}{2L_m} & -\frac{R_m}{2L_m} & -\frac{R_m}{2L_m} \\ 0 & 0 & \frac{r_m}{2} & -\frac{r_m}{2} & 0 & 0 \\ 0 & 0 & \frac{-r_m(b_1-b_2)}{2J_{eq}} & \frac{r_m(b_1-b_2)}{2J_{eq}} & \frac{K_m r_m}{2J_{eq}} & -\frac{K_m r_m}{2J_{eq}} \\ 0 & 0 & -\frac{K_m}{2L_m} & \frac{K_m}{2L_m} & -\frac{R_m}{2L_m} & \frac{R_m}{2L_m} \end{bmatrix}$$

$$P^{-1}AP = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & -\frac{b_1+b_2}{J_{eq}} & \frac{r_m K_m}{J_{eq}} & 0 & 0 & 0 \\ 0 & -\frac{K_m}{r_m L_m} & -\frac{R_m}{L_m} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & \frac{-(b_1-b_2)}{J_{eq}} & \frac{K_m r_m}{J_{eq}} \\ 0 & 0 & 0 & 0 & -\frac{K_m}{r_m L_m} & -\frac{R_m}{L_m} \end{bmatrix}$$

$$B^T = \begin{bmatrix} 0 & 0 & 0 & 0 & \frac{1}{L_m} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{L_m} \end{bmatrix}$$

$$P^{-1}B\begin{bmatrix} u_l \\ u_r \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ \frac{1}{2L_m} & \frac{1}{2L_m} \\ 0 & 0 \\ 0 & 0 \\ \frac{1}{2L_m} & -\frac{1}{2L_m} \end{bmatrix}\begin{bmatrix} u_l \\ u_r \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \frac{u_l+u_r}{2L_m} \\ 0 \\ 0 \\ \frac{u_l-u_r}{2L_m} \end{bmatrix}$$

$$u_x = \frac{u_l + u_r}{2}, \quad u_y = \frac{u_l - u_r}{2}$$

The final decoupled matrices are as shown below:
Horizontal System:

$$\begin{bmatrix} \dot{x} \\ \ddot{x} \\ \frac{di_x}{dt} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & \frac{b_1+b_2}{J_{eq}} & \frac{r_m K_m}{J_{eq}} \\ 0 & -\frac{K_m}{L_m r_m} & -\frac{R_m}{L_m} \end{bmatrix}\begin{bmatrix} x \\ \dot{x} \\ i_x \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \frac{1}{L_m} \end{bmatrix} u_x$$

Vertical System:

$$\begin{bmatrix} \dot{y} \\ \ddot{y} \\ \frac{di_y}{dt} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & \frac{-(b_1-b_2)}{J_{eq}} & \frac{r_m K_m}{J_{eq}} \\ 0 & -\frac{K_m}{L_m r_m} & -\frac{R_m}{L_m} \end{bmatrix}\begin{bmatrix} y \\ \dot{y} \\ i_y \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \frac{1}{L_m} \end{bmatrix} u_y$$

1.4. Drawing Shapes (Q1.4)

For this problem, a number of input signal equations were utilized to draw two different shapes. The first is an ellipse, and the second is a heart. The heart is chosen to demonstrate a shape with sharp edges. For both shapes, the following parameter values are chosen:

Parameter	Symbol	Value	Unit
Motor pulley radius	r_m	0.006	m
Motor inductance	L_m	0.003	H
Motor resistance	R_m	3	Ω
Motor inertia	J_m	6×10^{-6}	kg·m²
Motor constant	K_m	0.05	Nm/A
Print-head mass	m	0.15	kg
Print area x-dimension	d_x	0.35	m
Print area y-dimension	d_y	0.35	m
Rolling bearing friction	μ_r	0.05	Ns/m
Linear bearing friction, x	$\mu_{l,x}$	0.05	Ns/m
Linear bearing friction, y	$\mu_{l,y}$	0.05	Ns/m

The motor current inputs will be found by rearranging the motor equations from Q1.2 as shown below:

$$i_x = \frac{J_{eq}\ddot{x} + b_x\dot{x}}{r_m K_m} \tag{1.38}$$

where

$$b_x = b_1 + b_2, \qquad b_y = b_1 - b_2 \tag{1.39}$$

Substituting the current and its derivative into the electrical equation exposes the horizontal input voltage as shown below:

$$u_x = \frac{L_m J_{eq}}{r_m K_m}\dddot{x} + \frac{L_m b_x + R_m J_{eq}}{r_m K_m}\ddot{x} + \frac{R_m b_x + K_m^2}{r_m K_m}\dot{x} \tag{1.40}$$

The u_y equation is calculated in a similar way to become:

$$u_y = \frac{L_m J_{eq}}{r_m K_m}\dddot{y} + \frac{L_m b_y + R_m J_{eq}}{r_m K_m}\ddot{y} + \frac{R_m b_y + K_m^2}{r_m K_m}\dot{y} \tag{1.41}$$

These inputs are converted into the individual left- and right-motor voltages using

$$u_{m,l} = u_x + u_y$$

$$u_{m,r} = u_x - u_y$$

1.4.1. Ellipse

The following ellipse equations were used for the simulation:

$$x(t) = c_x + a\cos(\omega t) \tag{1.42}$$

$$y(t) = c_y + b\sin(\omega t) \tag{1.43}$$

Where c_x & c_y are the center x & y points of the shape, a and b are 1/2 the height and width. The angular frequency ω defines the printing speed. The chosen values are defined below:

c_x = 0.15
c_y = 0.15
a = 0.05
b = 0.03
$\omega = \pi rad/s$

Two graphs were made for each shape to visualize the inputs and positions. Then, the drawn shape was plotted.

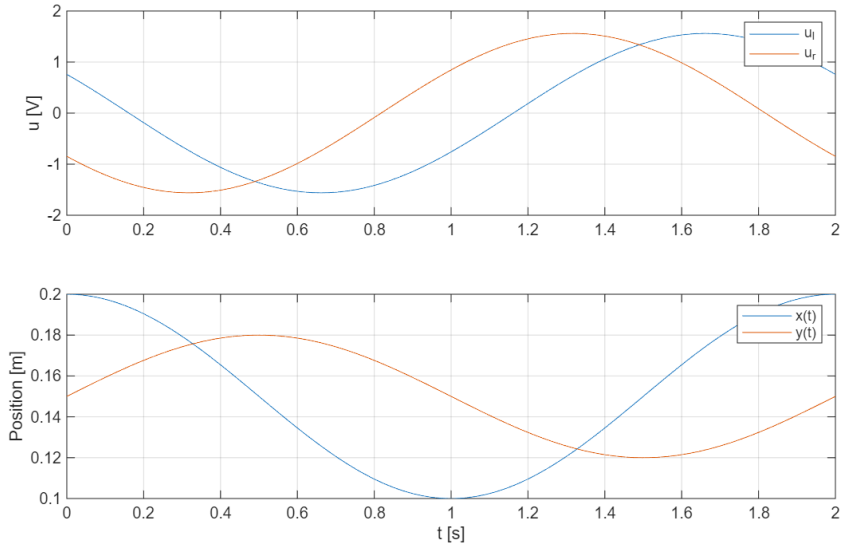


Figure 1.1: Ellipse input (a) and position (b) graphs

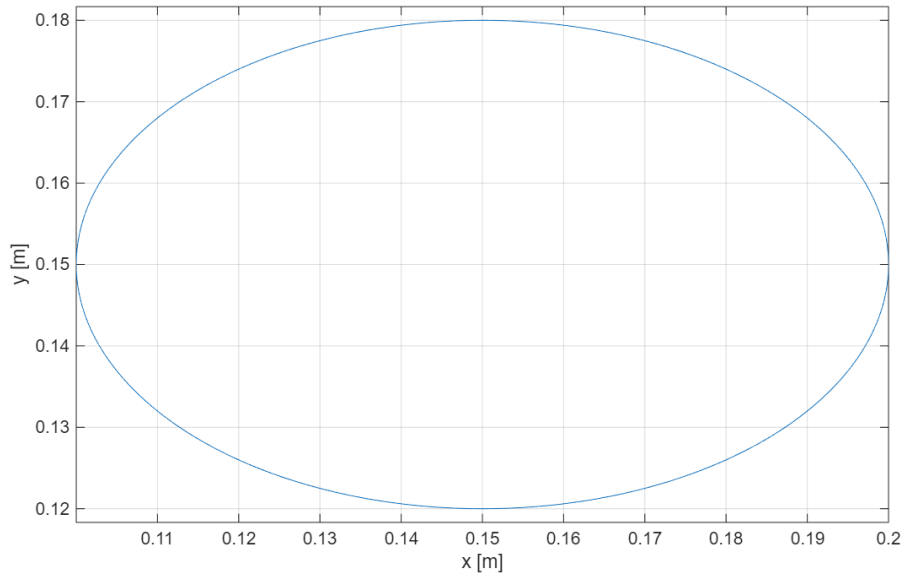


Figure 1.2: Ellipse plot

1.4.2. Heart

The heart utilized the following trajectory equations:

$$x(t) = c_x + s(12sin(\omega t) - 4sin(3\omega t)); \quad (1.44)$$

$$y(t) = c_y + s(13cos(\omega t) - 5cos(2\omega t) - 2cos(3\omega t) - cos(4\omega t) + 2.5); \quad (1.45)$$

Where:
c_x = 0.15
c_y = 0.15
a = 0.05
b = 0.05
s (scaling factor) = 0.003

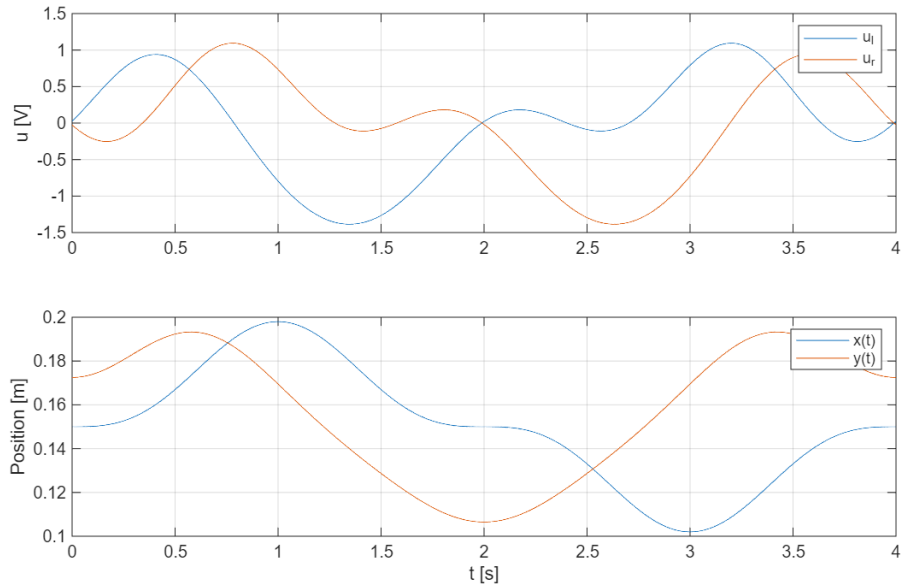


Figure 1.3: Heart input (a) & position (b) plots

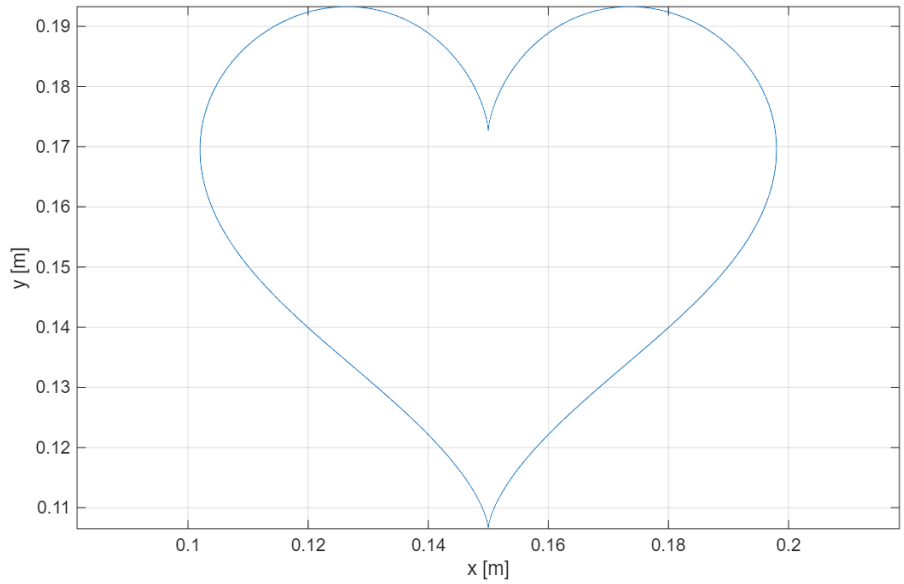


Figure 1.4: Heart plot

As can be seen in the heart input plot, the lines are much less sinusoidal compared to the ellipse plots. Also, the heart required multiple extra terms. This means that shapes with sharp edges are significantly more difficult to create.

1.5. Modelling the Nozzle (Q1.5)

The nozzle temperature is controlled using an electric heating circuit consisting of a voltage source, inductor, capacitor, and resistor. This resistor converts current into thermal energy and this resultingly increases the nozzle temperature. Heat is also lost to the surrounding environment through a thermal resistance.

The system is modelled by combining electric circuit equations into an energy balance equation. From this, the governing equations can be created. The variables which will be used are i_L for the inductor current, u_C for capacitor voltage, and L_n for nozzle inductance.

First, the equations for the various parts will be shown, after which they will be linked.

Important equations of this part

Capacitor:

$$i_c = C_n \frac{du_c}{dt} \tag{1.46}$$

Resistor:

$$u_R = R_n * i_r \tag{1.47}$$

Inductor:

$$u_L = L_n \frac{di_L}{dt} \tag{1.48}$$

The input is the controllable voltage u_n. This is related to resistor voltage u_R in the following way.

$$u_n = u_R + u_L \tag{1.49}$$

$$u_r = u_n - L_n \frac{di_L}{dt} \tag{1.50}$$

The same can now be done for the resistor current i_R, which is calculated in the following way. As this is a parallel circuit and not a circuit with all elements in series the Kirchhoff's first law can be applied

for the point between the resistor, inductor and capacitor.

$$i_L = i_c + i_R \tag{1.51}$$

Furthermore, a definition for current i_c was already given in 1.46 which can be switched out. Combined with this and the knowledge that due to being in parallel that $u_r = u_c$, this means that for this closed system the state variables are u_c, i_L, $\dot{u}_c$, and $\dot{i}_L$.

1.5.1. Linking the heat and electrical domains

The heat and electrical domains can be linked using the fact that any energy put into the system by the resistor is either stored in the filament or lost to the environment. This means that in this case the following equation holds:

$$u_r * i_r - W_r * T_n = W_c * T_n \tag{1.52}$$

As in the electrical domain, all parts have already been described and the equations for heat are already given. The system can now be described in the form of $\dot{x}_n = f_n(x_n, u_n)$.

Starting off with $\frac{di_L}{dt}$:

$$u_r = u_n - L_n \frac{di_L}{dt} \tag{1.53}$$

$$u_n - u_r = L_n \frac{di_L}{dt} \tag{1.54}$$

$$\frac{di_L}{dt} = \frac{u_n - u_r}{L_n} \tag{1.55}$$

And because $u_r = u_c$, the final equation comes down to:

$$\frac{di_L}{dt} = \frac{u_n - u_c}{L_n} \tag{1.56}$$

Continuing to $\frac{du_c}{dt}$

$$i_L = i_c + i_R \tag{1.57}$$

Because the inductor and source current are in series, they are equal. Using 1.46, the equation now changes to the following:

$$i_n = C_n * \frac{du_c}{dt} + i_R \tag{1.58}$$

$$i_n - i_R = C_n \frac{du_c}{dt} \tag{1.59}$$

$$i_n - u_r/R_n = C_n \frac{du_c}{dt} \tag{1.60}$$

Because $u_r = u_c$, the formula can be rewritten as:

$$\frac{du_c}{dt} = \frac{1}{C_n} * (i_n - \frac{u_c}{R_n}) \tag{1.61}$$

Lastly, the heat equation must be rewritten to find the equation for $\frac{dT_n}{dt}$.

1.5.2. Thermal Equations

Rewriting the electrical side to include only input or state variables, the following equation emerges:

$$P = u_r * i_r = \frac{u_r^2}{R_n} = \frac{u_c^2}{R_n} \tag{1.62}$$

Conservation of Energy:

$$\dot{Q}_{in} - \dot{Q}_{in} = \dot{E}_{stored} \tag{1.63}$$

Heat lost to the environment

$$\dot{Q}_{out} = \frac{T_n - T_e}{R_{t,n}} \tag{1.64}$$

Assuming the environmental temperature is constant, $dT_e/dt = 0$ and the stored thermal energy is:

$$\dot{E}_{stored} = C_{t,n}\frac{d(T_n)}{dt} \tag{1.65}$$

Substituting into the energy balance equation:

$$\frac{u_c^2}{R_n} - \frac{T_n - T_e}{R_{t,n}} = C_{t,n}\frac{dT_n}{dt} \tag{1.66}$$

Lastly, rounding off the final equation for $\frac{dT_n}{dt}$ comes out to:

$$\frac{dT_n}{dt} = \frac{1}{C_{t,n}}\left(\frac{u_c^2}{R_n} - \frac{T_n - T_e}{R_{t,n}}\right) \tag{1.67}$$

This equation describes how the nozzle temperature changes based on electrical heating and heat loss to the environment. The temperature increases when the power generated exceeds the heat loss.

1.5.3. State-Space Representation

The electrical and thermal equations can now be combined into a non-linear state-space representation. The state variables are defined as $x_1 = i_L$, $x_2 = u_C$, and $x_3 = T_n$, representing the inductor current, capacitor voltage, and nozzle temperature. The x-matrix of the state vector is then:

$$x_n^T = \begin{bmatrix} i_L & u_C & T_n \end{bmatrix} \tag{1.68}$$

Because the system contains a non-linear term u_C^2, the system can be expressed in the general non-linear state-space form:

$$\dot{x}_n = f_n(x_n, u_n) \tag{1.69}$$

$$\begin{bmatrix} \dot{i}_L \\ \dot{u}_C \\ \dot{T}_n \end{bmatrix} = \begin{bmatrix} 0 & -\frac{1}{L_n} & 0 \\ \frac{1}{C_n} & -\frac{1}{R_n C_n} & 0 \\ 0 & \frac{u_C}{R_n C_{t,n}} & -\frac{1}{R_{t,n} C_{t,n}} \end{bmatrix} \begin{bmatrix} i_L \\ u_C \\ T_n \end{bmatrix} + \begin{bmatrix} \frac{1}{L_n} & 0 \\ 0 & 0 \\ 0 & \frac{1}{R_{t,n} C_{t,n}} \end{bmatrix} \begin{bmatrix} u_n \\ T_e \end{bmatrix} \tag{1.70}$$

The state matrix utilizes the electrical and thermal dynamics, while the input matrix shows how the applied voltage and environmental temperature will change the system. The system remains non-linear due to the capacitor voltage, u_C, being included in the state variables list.

1.6. Printing a Circle of Filament (Q1.6)

1.6.1. Conservation of Energy

To begin modelling the printing of a circle of filament, the conservation of energy equation is used as shown below:

$$\dot{E}_{in} - \dot{E}_{out} = \dot{E}_{stored} \tag{1.71}$$

There are a few assumptions that can be made:

1. The heat is assumed to have longitudinal conduction and the net heat rate is $ka^2\frac{\partial^2 T}{\partial x^2}$
2. The heat rate entering from the plate is $h_b(a\Delta x)(T_b - T)$
3. The heat is lost to the surrounding air through the top and two lateral surfaces of the square filament, while the bottom is in contact with the bed and the end faces are represented by longitudinal conduction. The resulting convective heat loss is $3h_{air}(a\Delta x)(T - T_e)$

4. The rate of thermal energy change within the volume is $\rho a^2 \Delta x c_p \frac{\partial T}{\partial t}$

Thus, the energy equation becomes:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} + h_b/(\rho c_p a)(T_b - T) - 3h_{air}/(\rho c_p a)(T - T_e) \tag{1.72}$$

$\alpha = k/(\rho c_p)$ is the thermal diffusivity

1.6.2. Boundary Conditions

The starting time is defined as the moment the filament begins to print. The boundary is also assumed to move with the nozzle and it is assumed that temperature does not change with position. Thus, $\frac{\partial T}{\partial x} = 0$ during printing.

1.6.3. Discretization Scheme

The partial differential equation is solved numerically using an explicit Forward-Time Central-Space (FTCS) finite difference scheme. The spatial central difference is decomposed into intermediate physical heat flows and distributed sources.

The spatial domain is discretized into nodes separated by Δx, and the temporal domain into steps of Δt. The discrete constants are formulated from the thermal properties as follows:

$$C_1 = \alpha \frac{\Delta t}{\Delta x^2} \tag{1.73}$$

$$C_2 = \frac{h_b}{\rho c_p a} \Delta t \tag{1.74}$$

$$C_3 = \frac{3h_{air}}{\rho c_p a} \Delta t \tag{1.75}$$

The conductive heat transfer between adjacent spatial nodes $i - 1$ and i is calculated as an intermediate flow vector Φ:

$$\Phi_i = (T_{i-1}^n - T_i^n) C_1 \tag{1.76}$$

The distributed heat sources comprising of the heat gained from the bed and the heat lost to the ambient air are evaluated as a source vector f_T:

$$f_{T,i} = C_2(T_b - T_i^n) - C_3(T_i^n - T_e) \tag{1.77}$$

The future temperature at internal node i is updated by summing the net flow ($\Phi_{in} - \Phi_{out}$) and the distributed sources:

$$T_i^{n+1} = T_i^n + \Phi_i - \Phi_{i+1} + f_{T,i} \tag{1.78}$$

During the post-printing cooling phase, the elliptical shape closes into a continuous loop. This is modelled by utilizing periodic boundary conditions:

$$\Phi_{periodic} = (T_{N_{nodes}}^n - T_1^n) C_1 \tag{1.79}$$

1.6.4. Parameter Selection

The below values were chosen to simulate a standard PLA filament.

Parameter	Symbol	Value	Unit
Density (PLA)	ρ	1250	kg/m^3
Specific Heat (PLA)	c_p	1800	J/(kg·K)
Thermal Conductivity	k	0.13	W/(m·K)
Cross-section side	a	0.4×10^{-3}	m
Nozzle Temp	T_n	210	^{o}C
Bed Temp	T_b	60	oC
Environmental Temp	T_e	20	^{o}C
Free Convection	h_{air}	15	W/(m^{2}K)
Bed Contact	h_b	100	W/(m^{2}K)

Table 1.1: Thermal and physical parameters utilized for the 1D filament cooling simulation.

For numerical stability, an explicit FTCS scheme requires the Fourier number (represented here by C_1) to remain below 0.5. With the selected spatial step $\Delta x = 0.001$ m and temporal step $\Delta t = 0.01$ s, the discrete constant calculates to $C_1 \approx 5.78 \times 10^{-4}$, satisfying the stability criterion.

1.6.5. Simulation Execution and Results

Spatial nodes are activated at the extrusion temperature $T_n = 210^o$C as the nozzle moves along the shape. The transient model continues to evaluate until the maximum nodal deviation falls within 0.1^oC of the steady-state equilibrium. The theoretical steady-state uniform temperature (T_{eq}) is derived by setting the transient and spatial derivatives to zero to find the equilibrium point where heat flux from the bed equals the convective heat loss to the surrounding air:

$$T_{eq} = \frac{h_b T_b + 3h_{air} T_e}{h_b + 3h_{air}} = 47.59^o\text{C} \tag{1.80}$$

Running the decomposed flux simulation yields the following values:

- **Total required cooling time:** 46.28 s
- **Final uniform equilibrium temperature:** 47.59 ^{o}C

2

Discrete control

In this chapter the discrete control part of the 3D printer will be modelled. In this part a few different situations will be considered for which a finite state machine will be created.

2.1. Filament replacement

In case that the filament needs to be replaced for any reason it is important that the printer can keep printing correctly.

2.1.1. State codification

Before getting into state transitions multiple states must be codified. These very basically are: Inert, Printing, Filament replacement. As there is one procedure for both the case that a new colour is required and for when the spool runs out with only a differing input signal. Realistically there are other processes that happen during printing heating and cooling of both the bed and the nozzle for example. For this finite state machine they all fall under the process of printing. The state 'inert' was added as it seemed more realistic that a 3D printer does not immediately start printing when turned. In this state the printer does nothing yet, until it receives a file and then jumps to the printing state.

2.1.2. Final state machine for printing

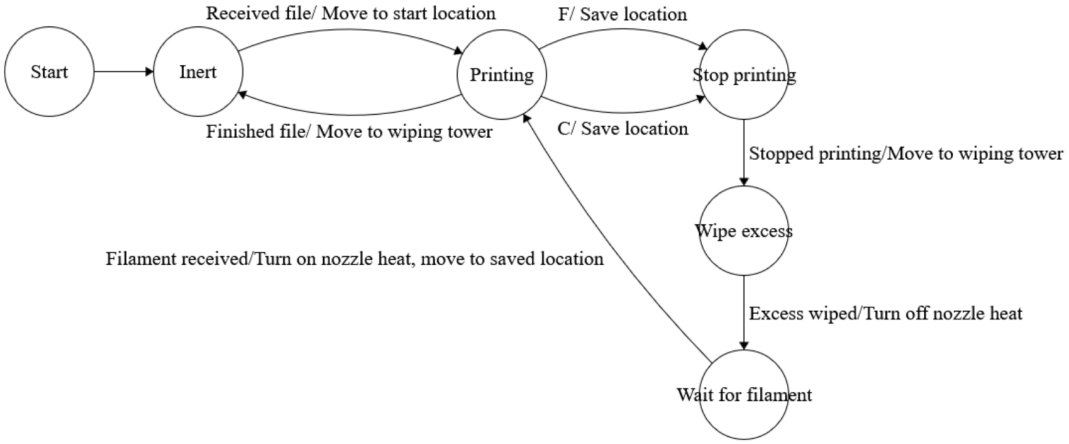


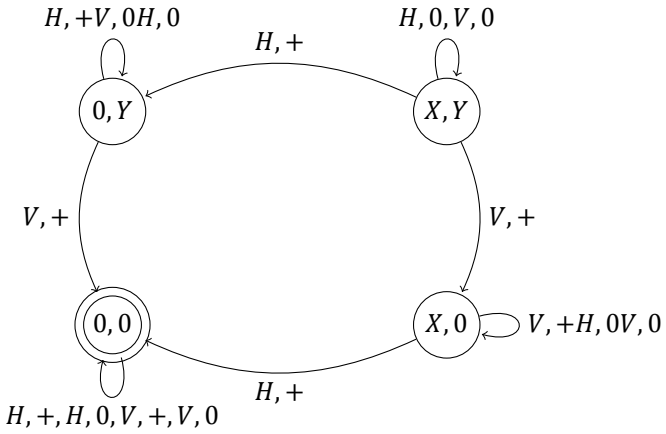
Figure 2.1: Discrete FSM of the 3D printer

The procedure for filament replacement is someone simplified with some skips being made. In the procedure, but this is because these rely upon actions which the 3D printer cannot undertake itself.

2.2. Parked detection

In this part an automaton will be built with which the the nozzle can be parked in the bottom left part of the printbed. For this there are a few options which can be done. Either there is a movement left (H) or a movement down (V) otherwise there is the choice of either having angular velocity (+) or not having angular velocity (0) for this any combination of them is possible. Now the goal is create an automaton from this. The automaton looks as follows:

- Q : (X,Y),(X,0),(0,Y),(0,0)
- Σ : (H,0),(H,+),(V,0),(V,+)
- δ : Are listed in the figure below
- Q_0 : (H,V)
- F: (0,0)



In this automaton X and Y mean any non zero distance from the left or bottom along its respective axis.

3
Hybrid system

3.1. Filament Replacement (Q3.1)

In this section, the systems from the previous two chapters are merged into a hybrid system. The hybrid system combines continuous and discrete states, allowing the physical behaviour of the printer to change depending on its current operating state, which includes printing, moving, etc. and will be expanded upon in more detail below.

3.1.1. Finite state machine

The hybrid system is implemented in MATLAB using the ode45 solver with event detection. The continuous equations are integrated until an event is detected, after which the FSM transitions to a new state. The physical model then continues under the conditions associated with the new state. The finite state machine associated with the full hybrid system looks similar to the final state machine from part 2.1 and is shown below:

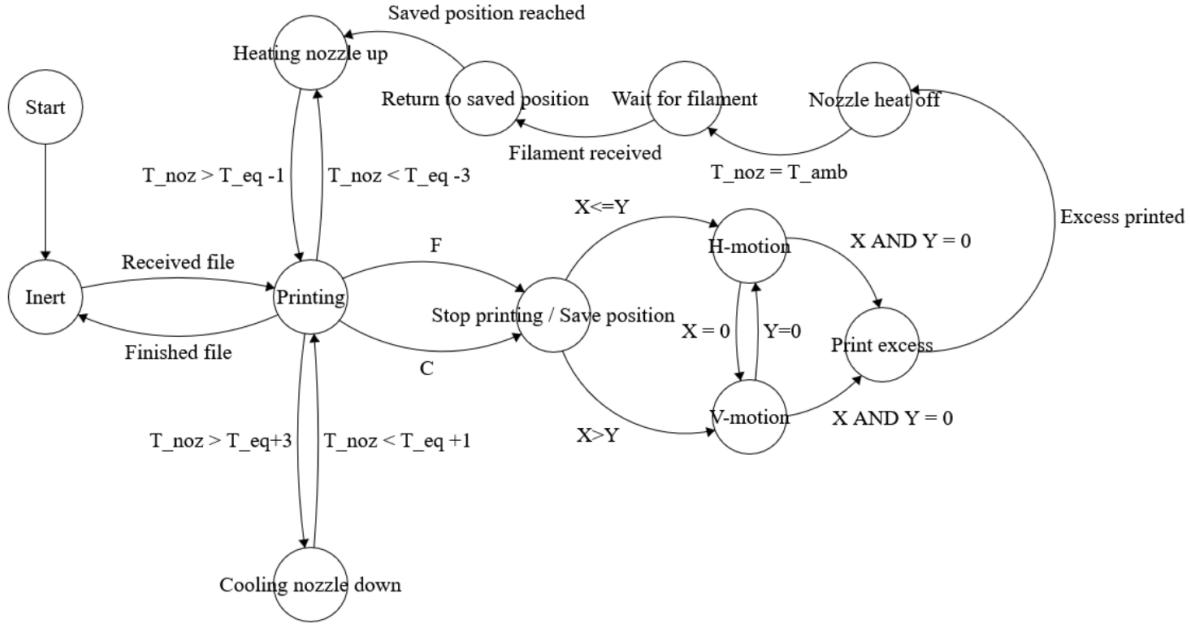


Figure 3.1: Finite state machine integrating

3.1.2. Continuous System Description

The continuous portion of the hybrid system combines the decoupled motor dynamics from Q1.3 with the nozzle electrical and thermal model from Q1.5 and has the goal of creating a circle of filament as was requested in Q1.6. The complete continuous state vector contains 9 variables and is defined as:

$$\mathbf{x}^T = \begin{bmatrix} x & \dot{x} & i_x & y & \dot{y} & i_y & i_L & u_C & T_n \end{bmatrix}$$

The first six states describe the position, velocity, and equivalent motor current in both directions. The final three states represent the nozzle inductor current, capacitor voltage, and temperature.

3.1.3. Discrete State Descriptions

There are a few discrete states which have to be considered for the system. There are various routes from which the system may follow, where it will transition only from one state to another specific state.

The state transitions were chosen in such a way that system behaviour would be logical and unnecessary state transitions would be avoided.

State	Description	Function
1	Inert	The printer remains stationary while waiting for the initial start command. This is also the final state once printing is completed.
2	Printing	The printer follows the curve set out in the programming while depositing filament. The motor voltages are calculated using the reference trajectory and feedback controller.
3	Filament replacement	Printing is interrupted when the roll is empty or the colour must be swapped. The current nozzle position is saved and the direction toward the wiping tower is determined. The type of movement the system makes is dependent on which axis is closer, e.g if the location of the printhead is (0.14 , 0.16) it will first move to (0, 0.16) then to (0,0).
4	Move to X-wall route	The nozzle moves toward $(X_{\log}, 0)$, maintaining its current X-coordinate while travelling toward the bottom wall.
5	Move to Y-wall route	The nozzle moves toward $(0, Y_{\log})$, maintaining its current Y coordinate while travelling toward the left wall.
6	Move to wiping tower	The nozzle moves from the selected wall location toward the wiping tower at $(0, 0)$.
7	Wipe excess filament	The nozzle remains at the wiping tower while excess filament is removed.
8	Nozzle heater off	The heater voltage is switched off, allowing the nozzle to cool.
9	Load new filament	The printer waits for a specified loading duration while the heater remains switched off.
10	Nozzle heater on	The heater is reactivated and the nozzle temperature increases until it reaches the reheating threshold.
11	Return to saved position	The nozzle travels back to the previously saved printing location before resuming the original trajectory.
12	Nozzle temperature high	Printing is interrupted if the nozzle temperature exceeds its upper limit. Heating is stopped until the temperature decreases sufficiently.
13	Nozzle temperature low	Printing is interrupted if the nozzle temperature falls below its lower limit. Heating is increased until the temperature recovers.

Table 3.1: Finite state machine states, descriptions, and functions.

3.1.4. Combined Hybrid System

The hybrid system uses discrete events to determine when transitions between operating states occur, as can be written as:

$$\dot{x} = f_q(x, u) \tag{3.1}$$

where q represents the discrete operating state. Three main events are considered **Event C - New filament colour**

The colour change is requested through the Boolean signal C. When this event is triggered at the defined time T_C, the printer interrupts its normal operation and enters the filament replacement state. **Event F - Filament R Out**

When the filament runs out, this event is triggered using the printing progress. The distance along the reference trajectory is calculated using:

$$s(t) = \int_0^{\tau} \sqrt{\dot{x}(\xi)^2 + \dot{y}(\xi)^2}\, d\xi \tag{3.2}$$

where τ is the effective printing time, excluding the interruptions. The effective printing time ensures that interruptions to not continue the printing trajectory and allows colour-changing and filament finishing events to happen in the same simulation if needed.

Event E - Excess Filament Removed When the nozzle reaches the wiping tower, the excess filament is removed, and this takes a specified 0.2 s.

3.1.5. Invariants Guards and Reset Maps

The continuous and discrete dynamics of the hybrid system are connected through invariants, guards, and reset maps. Invariants are used to define the conditions under which the system remains in a specific state. The guards define when a transition between states occurs. Reset maps keep track of which variables are updated when a transition takes place.

The variants and guards are used for the nozzle position, velocity, temperature, printing progress, and elapsed time. The guards and reset maps are shown in the table below.

Transition	Guard condition	Reset or update
$1 \rightarrow 2$	Start time reached	Begin printing
$2 \rightarrow 3$	Event C or F begun	Save current position and printing progress
$3 \rightarrow 4/5$	Compare current X and Y coordinates	Determine closest wall
$4/5 \rightarrow 6$	Closest wall reached	Set wiping tower as destination
$6 \rightarrow 7$	Wiping tower reached	Begin wiping
$7 \rightarrow 8$	Wiping completed (E)	Switch heater off
$8 \rightarrow 9$	Heater-off	Begin filament swapping
$9 \rightarrow 10$	Filament swapping completed	Turn on heater
$10 \rightarrow 11$	Nozzle temperature ≥ 208°C	Return to saved location
$11 \rightarrow 2$	Saved location reached	Resume printing
$2 \rightarrow 12$	Nozzle temperature ≥ 214°C	Stop printing and disable heating
$12 \rightarrow 2$	Nozzle temperature ≤ 211°C	Resume printing
$2 \rightarrow 13$	Nozzle temperature ≤ 206°C	Stop printing and increase heating
$13 \rightarrow 2$	Nozzle temperature ≥ 209°C	Resume printing
$2 \rightarrow 1$	Printing completed	End printing

Table 3.2: Invariant guard & reset map overview

3.1.6. Error Correction

In order to keep the printer head on track, feedback and feedforward controls are used. This is done with PD control and the relevant formulas are shown below:

$$u_x = u_{x,ff} + K_p(x_r - x) + K_d(\dot{x_r} - \dot{x}) \tag{3.3}$$

$$u_y = u_{y,ff} + K_p(y_r - y) + K_d(\dot{y_r} - \dot{y}) \tag{3.4}$$

3.1.7. Simulation Results

The hybrid model was simulated using an elliptical printing trajectory with a printing duration of 2s. The colour-change trigger was set to T_C = 0.50s and the filament-finished event was set to a reference path length of 0.23m. The system was able to complete both interruptions within the same simulation. The trajectory plot for the completed printing task including the replacement events is shown below:

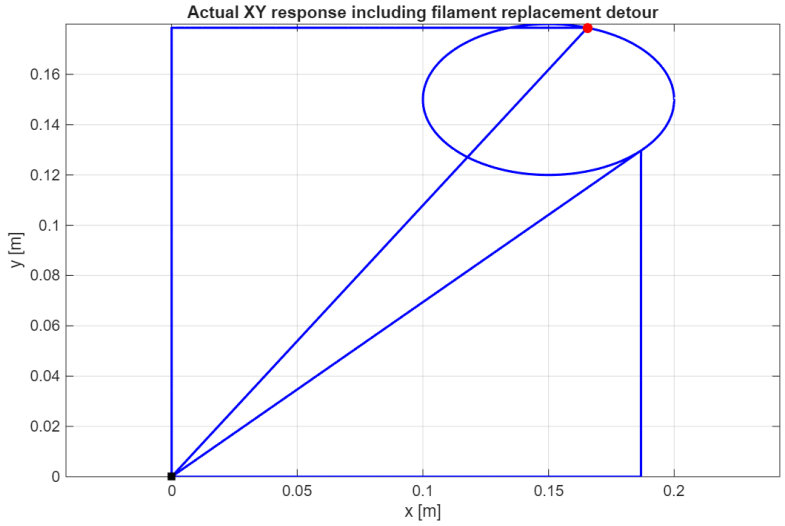


Figure 3.2: Trajectory of nozzle

Figure 3.2 illustrates the print-head motion during normal printing and filament replacement. The trajectory contains the elliptical path together with the additional movements towards the wiping tower. The red dot indicates the first replacement trigger and the print head returned to the saved location after pausing to continue the original trajectory.

A plot depicting the state of the system over time is shown below:

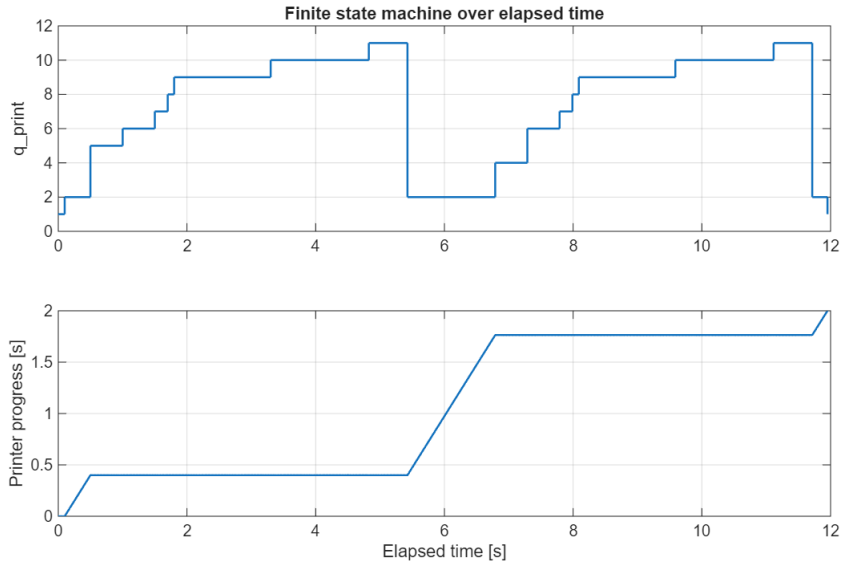


Figure 3.3: States of system over time

Figure 3.3 demonstrates the interaction between the discrete controller and the continuous printing process. The FSM passes through a replacement event twice as shown here. The printing progress graph (bottom) shows two periods during which progress remains constant while replacement takes place. After each interruption, the printing resumes from the saved progress value. Because the reference trajectory only requires 2 seconds of printing time, the total elapsed simulation time increases to 12s due to the additional operations.

Finally, a plot was created to illustrate how the filament heats and cools.

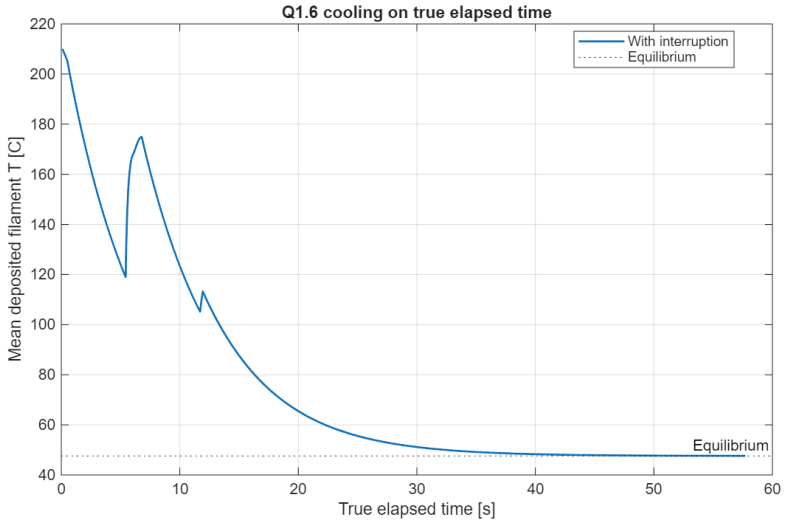


Figure 3.4: Filament Temperature over time

Figure 3.4 correctly shows that the deposited filament continues to cool while the printing is interrupted. The mean temperature decreases during the replacement operations and increases when new, hotter filament is deposited. Once the print is completed, the filament temperature approaches and eventually reaches the equilibrium temperature.

3.2. Drawing a Square (Q3.2)

In this section, the 3.1 model is extended to now draw a perfect square using the angular actuators. In order to do this, the sine and cosine equations used in 3.1 are not applicable. The best these equations could create is a square with rounded corners.

To generate the square, the trajectory is divided into 4 straight-line segments where the print head can move along one axis while the other coordinate remains constant.

In order to smoothly decelerate and re-accelerate at the corners, cubic polynomial trajectory planning is utilized and the formula for this is shown below:

$$r(t) = r_0 + (r_f - r_0(3\tau^2 - 2\tau^3)) \tag{3.5}$$

where r_0 and r_f represent the initial and final positions of the segment, and $\tau = t/T_{\text{side}}$ is the normalized time, ranging from 0 to 1. T_side is the specified time to print each side of the square. The position, velocity, acceleration, and jerk are calculated by this equation and are shown below.

$$\dot{r}(t) = \frac{r_f - r_0}{T_{\text{side}}}(6\tau - 6\tau^2) \tag{3.6}$$

$$\ddot{r}(t) = \frac{r_f - r_0}{T_{\text{side}}^2}(6 - 12\tau) \tag{3.7}$$

$$\dddot{r}(t) = -\frac{12(r_f - r_0)}{T_{\text{side}}^3} \tag{3.8}$$

These formulas can then be fed into the feedforward and feedback motor controllers.

3.2.1. Motor Angular Feedback

The print-head position and velocity are reconstructed using the kinematic relationships from Q1.1 and replace the direct Cartesian measurements in the PD feedback controller developed in Q3.1

$$\theta_l = \frac{x + y}{r_m}, \qquad \theta_r = \frac{x - y}{r_m} \tag{3.9}$$

$$x_{\text{enc}} = \frac{r_m}{2}(\theta_l + \theta_r), \qquad y_{\text{enc}} = \frac{r_m}{2}(\theta_l - \theta_r) \tag{3.10}$$

$$\dot{x}_{\text{enc}} = \frac{r_m}{2}(\dot{\theta}_l + \dot{\theta}_r), \qquad \dot{y}_{\text{enc}} = \frac{r_m}{2}(\dot{\theta}_l - \dot{\theta}_r) \tag{3.11}$$

3.2.2. Simulation Results

An XY trajectory plot is used to visualize the actual path taken by the print head as shown below:

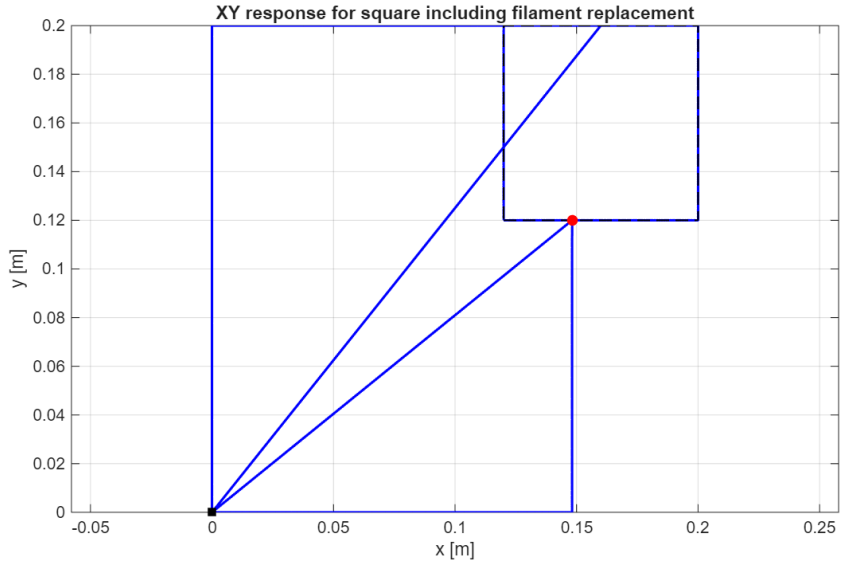


Figure 3.5: Square XY Trajectory

As can be seen, the print head follows the trajectory of a perfect square and show the replacement events.

The next figure shows how the motor angular motion changes over time.

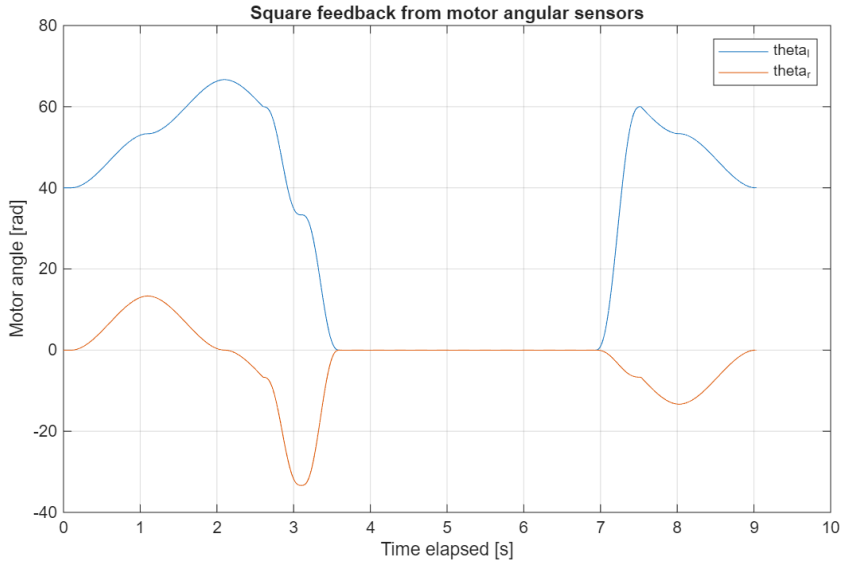


Figure 3.6: Motor angular motion over time

3.3. AI statement

In question 1, AI was used to find online website or video sources which could better explain the concepts needed to answer each sub-question, as well as pinpointing where in the lecture slides necessary information was located and why it was important. In problem 3, AI was used to find Matlab documentation which might help with what we were needing to program and explain the functions/structure when necessary. Before submitting the report, AI was used to check for small typos or mistakes in the write-up. All AI-generated suggestions were carefully reviewed and validated before usage. The use of AI remained strictly supportive, aimed at improving clarity and readability, without replacing our own modelling and reasoning.